

facematch — usage guide

Drop-in Android library for live face verification. Hosts its own camera screen, returns a `FaceMatchOutcome` to the calling Activity via the `ActivityResultContract` API.

1. Add to your project

Unzip the `facematch/` folder so it sits next to your `app/` module.

`settings.gradle.kts`

```
include(":facematch")
```

`app/build.gradle.kts`

```
dependencies {  
    implementation(project(":facematch"))  
}
```

Sync Gradle. Done.

2. Project prerequisites

Your app's `build.gradle.kts` must satisfy:

```
android {  
    compileSdk = 36                                // or 35+  
    defaultConfig { minSdk = 24 }                  // hard floor – library is 24  
    compileOptions {  
        sourceCompatibility = JavaVersion.VERSION_17  
        targetCompatibility = JavaVersion.VERSION_17  
    }  
    kotlinOptions { jvmTarget = "17" }  
    buildFeatures { compose = true }  
}
```

The library is Compose-only. Your app must already declare the standard Compose stack (`compose-bom`, `compose.ui`, `material3`, `activity-compose`). Any production Compose app already has these.

The library declares `<uses-permission android:name="android.permission.CAMERA"/>` and registers `FaceMatchActivity` in its own manifest — both merge into your app automatically. You do not need to add either.

3. Quick start

```
import com.countrydelight.facematch.api.FaceMatchContract
import com.countrydelight.facematch.api.FaceMatchInput
import com.countrydelight.facematch.api.FaceMatchOutcome
import com.countrydelight.facematch.engine.FaceMatchConfig

class YourActivity : ComponentActivity() {

    private val faceMatchLauncher = registerForActivityResult(FaceMatchContract()) {
outcome ->
        when (outcome) {
            is FaceMatchOutcome.Matched -> uploadAttendance(outcome.croppedFacePath)
            is FaceMatchOutcome.Rejected -> showOtpFallback(outcome.croppedFacePath)
            FaceMatchOutcome.Cancelled -> {}
        }
    }

    private fun startFaceMatch(referenceEmbeddingBase64: String) {
        faceMatchLauncher.launch(
            FaceMatchInput(
                referenceEmbeddingBase64 = referenceEmbeddingBase64,
                config = FaceMatchConfig(), // optional – see §5
            )
        )
    }
}
```

You must request the `CAMERA` permission at runtime before launching:

```
val permissionLauncher = registerForActivityResult(
    ActivityResultContracts.RequestPermission()
) { granted -> if (granted) startFaceMatch(myStoredEmbedding) }

permissionLauncher.launch(Manifest.permission.CAMERA)
```

4. Outcomes

```
sealed interface FaceMatchOutcome {
    data class Matched(val croppedFacePath: String) : FaceMatchOutcome
    data class Rejected(val croppedFacePath: String?) : FaceMatchOutcome
    data object Cancelled : FaceMatchOutcome
}
```

Outcome	When it fires	croppedFacePath
Matched	User confirmed a successful live match via the "Done" button	JPEG of the matched face frame in your app's <code>cachedDir</code>
Rejected	Match failed (mismatch or spoof) and user exhausted <code>maxRetries</code>	JPEG of the last face frame, or <code>null</code> if none was captured
Cancelled	User pressed the back button on the verification screen	n/a

Spoof rejection (printed photo, screen replay, mask) is **routed through Rejected** — the consumer does not distinguish it from a normal mismatch. Internally the spoof detector still runs and contributes to the rejection.

5. Configuration

All fields are optional; defaults are tuned for production use.

```
FaceMatchConfig(
    matchThreshold = 72f,           // % cosine-similarity required to count a frame as
    matched
    triggerFrames = 8,             // consecutive matched frames before firing success
    rejectFrames = 20,             // consecutive sub-threshold frames before reject
    skipFrames = 2,               // MobileFaceNet runs every Nth qualified frame
    spoofRejectThreshold = 25,     // consecutive spoof frames before reject
    spoofEnabled = true,          // false = disables anti-spoof entirely
    maxRetries = Int.MAX_VALUE,   // "Retry" taps the rejection dialog allows
)
```

Tighten `maxRetries` (e.g. `3`) for production flows that should fall through to an OTP fallback after a few attempts.

6. Enrolling a reference face

The library compares a live face against a reference embedding (192-dim float vector, base64-encoded). To enroll a user, compute the embedding once from a captured face bitmap and store it on your backend / locally:

```
import com.countrydelight.facematch.engine.MobileFaceNetHelper

val helper = MobileFaceNetHelper(context)
val embedding = helper.getEmbedding(faceBitmap) ?: return // null on failure
val base64 = MobileFaceNetHelper.embeddingToBase64(embedding)
// Persist `base64`. Pass it as referenceEmbeddingBase64 on next verification.
helper.close()
```

`faceBitmap` should be a tight crop of a single, frontal face. Use ML Kit Face Detection (already a transitive dependency of this library) to detect and crop.

The base64 string is exactly **1024 characters** (192 floats × 4 bytes × 4/3 base64 expansion). If you paste a value of a different size, every match frame will fail with a logged "Embedding size mismatch" — no crash, just no match.

7. Branding (theme override)

The library ships with a default palette. To override before launching the contract:

```
import com.countrydelight.facematch.ui.theme.FaceMatchColors
import com.countrydelight.facematch.ui.theme.FaceMatchTheme

// Once at app startup, e.g. in Application.onCreate
FaceMatchTheme.defaults = FaceMatchColors(
    primary = Color(0xFF8E24AA),
    success = Color(0xFF388E3C),
    bottomBar = Color(0xFF1A237E),
    // ... any subset of slots
)
```

The Activity reads `FaceMatchTheme.defaults` on launch. Per-launch customization is not supported via the contract — set the static once.

To use the library composables directly (skipping the Activity), wrap them:

```
FaceMatchTheme(colors = FaceMatchColors(primary = brandBlue)) {
    LiveFaceMatchView(
        referenceEmbedding = embedding,
        onMatched = { ... },
        onRejected = { ... },
        onCancelled = { ... },
    )
}
```

8. What ships in the APK

The library brings these transitively:

- TensorFlow Lite (~2 MB) — MobileFaceNet 192-dim embeddings, MiniFASNet spoof
- ML Kit Face Detection (~3–5 MB after Play Services merging)
- CameraX (camera2, lifecycle, view)
- Lottie Compose — for the success animation
- Two `.tflite` assets totaling ~25 MB

Total APK growth: roughly **25–30 MB**. The `.tflite` files cannot be removed — they are the inference models.

9. Known caveats

- **Min SDK 24** — your app's `minSdk` cannot go below 24. Manifest merger will block lower values.
- **Portrait only** — `FaceMatchActivity` is locked to portrait orientation.
- **No state restoration** — the verification flow does not survive configuration changes. Rotation forces a restart; we lock portrait to avoid it.
- **Camera permission must be granted *before* launching the contract.** The Activity does not request it on your behalf.
- `MobileFaceNetHelper(context)` **is heavy to construct** (loads a 25 MB model). Cache it across enrollments; always call `.close()` when done.
- **Embedding format is strict.** The base64 must decode to exactly 768 bytes ($192 \times \text{float32}$). Anything else fails every match frame.

10. Verifying integration

After wiring up:

```
./gradlew :app:compileDebugKotlin
```

Then install on a device, grant `CAMERA`, tap your launch button. The verification UI should appear within a second. With the wrong reference embedding, you'll cycle through to `FaceMatchOutcome.Rejected` after `rejectFrames + maxRetries × rejectFrames` frames. With the right one, the success dialog appears once `triggerFrames` consecutive frames clear `matchThreshold`.